

Unit 17

Reporting & Professional Communication + Capstone Kickoff

Module 5 — Putting It Together · Week 17

The client isn't buying the hack. They're buying **the report**.

Where we are

- For 16 weeks you learned **how** to find and demonstrate weaknesses.
- Now you learn the part that **pays the bills**: the report.
- A brilliant test with a sloppy report is a **failed engagement**.
- Then we **kick off the capstone** — the project you finish and report on next week.

Clients pay for the document that tells them what's wrong, how bad it is, and how to fix it.

What you'll be able to do

By the end of this unit you can:

- Explain **why the report is the real deliverable** of a pentest.
- Identify the standard report sections: **exec summary, methodology, findings, remediation, appendices**.
- Write the same issue for **two audiences**: executives and technical staff.
- Assign a **severity** (Low/Med/High/Critical) and justify it with **likelihood × impact**.
- Document a finding with **evidence, impact, and specific remediation**.

What you'll be able to do (cont.)

- Apply the **penetration-test report rubric** to evaluate and improve a section.
- Explain the **ethics of accurate reporting** — no exaggerating, no minimizing, no hiding.
- Produce a **complete sample report** from a prior lab.
- Receive and confirm your **capstone target assignment**.

Vocabulary — the document

Term	Meaning
Deliverable	What the client actually receives and pays for — here, the report.
Penetration-test report	Formal doc: what was tested, found, how serious, how to fix.
Executive summary	Short, plain-language overview for busy, non-technical leaders.
Methodology	The section explaining <i>how</i> the test was done.
Appendix	Supporting material at the end (raw output, host lists, references).

Vocabulary — the finding

Term	Meaning
Finding	A single security issue, written up with evidence, severity, impact.
Evidence	Proof a finding is real — labeled screenshots, commands, hosts/URLs.
Severity / risk rating	How serious it is — Low / Medium / High / Critical.
Impact	What could actually go wrong for the organization.
Likelihood	How easy or probable it is that an attacker exploits it.
Remediation	The specific, actionable fix recommended.

Vocabulary — risk & communication

Term	Meaning
Recommendation	Broader advice to improve security beyond one fix.
CVSS	Common Vulnerability Scoring System — turns factors into a 0-10 score.
Audience	Who you're writing for — executives vs. technical staff differ.
Scope	The agreed boundary of what was authorized — restated in the report.
Accuracy	Reporting truthfully — not exaggerated, not minimized, not hidden.
Capstone	The end-of-course project: complete a CTF target, report, present.

Why this matters

- Every other unit taught a **skill** — this one teaches the **product**.
- In a real job, you may spend **half** your time writing.
- The report is what a client **keeps, shares, and acts on** for months.
- It outlives the test — auditors and new staff read it later.

The hack is a few days. The report is the part that lasts.

How this connects

- Units 7–15: you **found and proved** weaknesses.
- This unit: you **communicate** them so they get fixed.
- Next unit (capstone): you do **all of it** end to end.

You already did the hard part. Now make it count on paper.

Ethics & Authorization

This week's thread is **accuracy**. Two failures are equally serious.

The two ways to betray trust

- **Exaggerating** a finding to look impressive — scaring a client into wasting money, or crying wolf.
- **Minimizing or hiding** a finding because it's awkward, you ran out of time, or you like the client.
- Both **distort the truth the client paid for**.

A professional reports what is true — with evidence — and rates it honestly.

Scope lives in the report too

- The report **restates the scope and authorization** of the test.
- It is the **written record** proving the test was permitted and stayed in bounds.
- This is both professional practice and the ethics thread of the whole course.

The report is the document a client makes real decisions and spends real money on.

Day 1

Why the Report Is the Real Product + Report Anatomy

Warm-up

A tester spends two weeks brilliantly breaking into a client's network, then hands over a **one-paragraph email**.

Did the client get what they paid for? Why not?

The big idea

- Clients **pay for the report, not the hack**.
- A finding that isn't clearly written and reproducible **effectively doesn't exist** for the client.
- In industry, the report **is** the job.

"The hacking was the fun part" — true, but the report is the product.

Anatomy of a pentest report

Section	Who reads it	What it answers
Executive summary	Leaders	How exposed are we? What first?
Methodology	Technical reviewers	How did you test?
Findings	IT / fixers	What's wrong, with proof
Remediation	IT / leaders	How do we fix it?
Appendices	Technical staff	Raw output, references

The executive summary

- The **first** page, but often written **last**.
- Plain language — a busy leader reads it in **two minutes**.
- Answers: *How exposed are we? What should we fix first?*
- No commands, no jargon, no acronyms left unexplained.

If the CEO only reads one page, this is it.

Methodology

- Explains **how** you tested, in phases.
- recon → scan/enumerate → exploit → (post) → document.
- Names the **tools** used (nmap, gobuster, etc.).
- Lets a reviewer trust — and repeat — your work.

It shows the test was thorough and professional, not lucky.

Appendices

- The "**show your work**" section at the back.
- Raw `nmap` output, full host lists, long command logs.
- Links: CVE pages, OWASP references, a CVSS score if used.

Keeps the findings clean while still proving everything.

Check your understanding

A client says: *"Just send me the screenshots — skip the writing."*

Why is that a bad deal for the client?

Answer

- Screenshots alone show **what** but not **how bad** or **how to fix**.
- A leader can't prioritize; a developer can't reproduce.
- The **report turns proof into action** — that's the value.

Evidence without context is a puzzle, not a deliverable.

Guided practice (Day 1)

Sort each scrambled item into the right section:

- A labeled screenshot → **Findings (evidence)**
- A CEO-friendly paragraph → **Executive summary**
- Raw `nmap` output → **Appendices**
- A fix recommendation → **Remediation**
- A risk rating → **Findings (severity)**

Then begin **Lab Part A**: pick a prior-unit finding and fill in the basic facts.

Exit ticket: Name the five sections; which one does a non-technical CEO read first?

Day 2

Findings, Evidence, Severity & CVSS Awareness

Warm-up

Two versions of the same finding:

- A) A labeled screenshot + exact steps to reproduce.
- B) "The site is hackable."

Which can a developer actually **act on**?

Anatomy of a single finding

Every finding needs:

- **Title** — clear and specific (not "the website is bad").
- **Affected asset** — host, IP, or URL.
- **Evidence** — labeled screenshots, exact commands, URLs.
- **Severity** — Low / Med / High / Critical.
- **Impact** — what an attacker could actually do.
- **Remediation** — the specific fix.

If a reader can't reproduce it from your write-up, it isn't done.

Labeling evidence

Figure 1: login form accepts ' OR 1=1 -- (authentication bypass)

- Every screenshot gets a **figure number and caption**.
- Commands are written **exactly** so a reader can repeat them.
- Re-teach trigger: "trust me, it worked" with no evidence.

Good reports come from good lab journals. You can only report what you wrote down.

Severity = likelihood × impact

- **Likelihood:** how *easy* is it to pull off?
- **Impact:** how *bad* is it if they do?
- Multiply them in your head: high × high = top of the scale.

Severity ≈ Likelihood (how easy) × Impact (how bad)

A scary impact that's nearly impossible to trigger is **not** Critical.

The four severity levels

Rating	Rough meaning	Example
Critical	Easy AND severe (full system / data)	Unauth RCE; SQLi dumping the user table
High	Serious; some effort or a condition	Stored XSS stealing admin sessions
Medium	Real risk, limited or harder	Reflected XSS needing a click
Low	Minor; little direct impact	Verbose error leaking a version

Watch out for inflation

- The #1 student mistake: marking **everything** Critical.
- A real Critical is **rare** — full takeover, customer data.
- Inflated ratings make a client **stop trusting** your report.

If everything is an emergency, nothing is. Rate honestly.

Worked example — Critical

Finding: The login page is vulnerable to SQL injection that returns the full user table.

- **Likelihood:** high — one crafted request, no login needed.
- **Impact:** high — every customer record exposed.
- **Severity: Critical.** Easy *and* severe.

Worked example — Low

Finding: An error page prints the web server's exact version number.

- **Likelihood:** high — anyone can trigger the error.
- **Impact:** low — it only *hints* at what to attack next.
- **Severity: Low.** Easy but barely harmful on its own.

Check your understanding

Finding: A reflected XSS bug fires only if a victim clicks a crafted link you email them.

What severity, and why?

Answer

- **Likelihood:** medium — it needs a victim to **click** a special link.
- **Impact:** real but limited — affects whoever clicks.
- **Severity: Medium.** The required click lowers likelihood.

Same bug "stored" (auto-fires for everyone) would rate higher.

CVSS — awareness only

- **CVSS** = an industry method that turns factors into a **0-10 score**.
- It considers things like **how easy** the attack is and **how much damage** it does.
- You should know **it exists** and **roughly what feeds it** — not compute vectors by hand.

The online calculator is an extension task, not a requirement.

Guided + independent practice (Day 2)

In pairs, rate and **justify** three findings:

- Default admin password
- Reflected XSS
- Missing security header

Then **Lab Part B:** write your finding(s) with evidence and a justified severity.

Exit ticket: What three things must every finding include? Rate one example and justify it in a sentence.

Day 3

Writing for Two Audiences

Warm-up

Explain SQL injection to (a) your **principal** and (b) a **database administrator**.

How does the language change?

The two-audience problem

	Executive summary	Technical finding
Reader	Busy, non-technical leader	Staff who will fix it
Language	Plain, risk-focused	Precise, reproducible
Contains	"How exposed? What to do?"	Exact param, payload, URL, steps
Avoids	Commands, un-glossed acronyms	Vagueness

Same truth, different **altitude**.

Same finding, two ways

Technical finding:

Unsanitized input in the login form permits UNION-based SQL injection on `/login`.

Executive summary version:

An attacker could currently access customer records through the login page. We recommend prioritizing a fix this quarter.

The altitude idea

- Think of writing at different **altitudes**.
- **High altitude (executive)**: the whole landscape — risk and priority.
- **Ground level (technical)**: the exact path, step by step.
- Same map, different zoom — and **same truth**.

You don't change the facts. You change the focus.

Check your understanding

Which audience needs the exact URL, parameter, and payload to do their job?

Executives or technical staff?

Answer

- **Technical staff** — they fix the bug and must reproduce it.
- Executives need risk and priority, not commands.
- Give each reader exactly what helps *them* act.

Right detail, right reader. That's the whole skill.

Rules for the executive summary

- **No commands.** No `sqlmap`, no payloads, no code.
- **No un-glossed acronyms.** Spell it out or explain it in plain words.
- 2–4 sentences answering: *how exposed are we, and what should we do?*
- A good test: could the **principal** read it and know how worried to be and what to prioritize?

This is the hardest skill. Jargon leaks. Have peers hunt for it.

Guided + independent practice (Day 3)

- Take your Day-2 technical finding; draft a **2-3 sentence executive-summary version** — no jargon.
- Peer-swap and **flag any jargon** that slipped through.
- Then **Lab Part C:** write the executive summary + the remediation/recommendations.

Exit ticket: Rewrite for an executive: *"Unsanitized input in the login form permits UNION-based SQL injection."*

Remediation must be specific

Vague (not OK)	Specific (OK)
"Make it more secure"	"Use parameterized queries for all DB calls"
"Fix the login"	"Validate input server-side; reject bad characters"
"Patch it"	"Change the default admin password; enforce a policy"

Remediation a developer can act on **today**.

A fix vs. a recommendation

- **Remediation** = the precise fix for **this one** finding.
- **Recommendation** = bigger-picture advice for the whole org.
- Example fix: "Use prepared statements on `/login`."
- Example recommendation: "Add input-validation training for devs."

Fix the hole today; recommend how to stop the next one.

Check your understanding

A report says: *"Recommendation: improve security posture across all systems."*

What's wrong with it as remediation?

Answer

- It's **vague** — no one knows what to *do* on Monday.
- It names no system, no setting, no concrete step.
- Good remediation is **specific and actionable**.

"Improve security" is a wish, not a fix.

Day 4

Assemble the Full Report + Peer Review

Warm-up

Read the **report rubric** aloud.

Which row do you think you'll **lose the most points** on, and how will you fix that today?

Professional polish

- **Restate scope/authorization** at the top.
- **Label every screenshot** (Figure 1, 2, 3...).
- **Number your findings.**
- Use **consistent terminology** throughout.
- Add an **appendix** for raw tool output.

Reports come from notes — thin lab journals bite you here.

The report template

```
# Penetration Test Report – [Target / Lab Name]
**Tester:** ... **Date:** ...
**Scope & authorization:** authorized test of [target], Unit __, with permission.

## 1. Executive summary      (plain language, no jargon)
## 2. Methodology            (recon → scan → exploit → document)
## 3. Findings                (title, asset, severity+justification, evidence, impact)
## 4. Remediation & recommendations
## 5. Appendices              (raw output, references)
```

Worked report — header

```
# Penetration Test Report — Unit 12 Web Lab  
Tester: A. Student    Date: 2026-05-01  
Scope & authorization: Authorized test of the Unit 12  
practice web app, with instructor permission. No other  
systems were tested.
```

Start every report by proving it was **authorized and in scope**.

Worked report — executive summary

1. Executive summary

The login page lets an attacker read the entire customer list without a password. This is our most urgent issue; we recommend fixing it this week before any other work.

- Plain language. No `sqlmap`, no payloads, no acronyms.

A principal could read this and know how worried to be.

Worked report — methodology

```
## 2. Methodology
```

```
We tested in phases: recon, then scanning with nmap,  
then enumerating pages with gobuster, then manually  
testing inputs. All work stayed within the approved scope.
```

Short, honest, and shows the work was systematic.

Worked report — the finding

```
## 3. Findings
```

```
### Finding 1 — SQL injection on the login form
```

- Affected: `https://lab.local/login`
- Severity: Critical — easy to run, exposes all user data
- Evidence: Figure 1 — input `' OR 1=1 --` returns all rows
- Impact: An attacker could read every customer record.

Title, asset, justified severity, evidence, impact — all five.

Worked report — remediation

4. Remediation & recommendations

- Finding 1 fix: Use parameterized queries on all database calls; validate input server-side.
- Recommendation: Add secure-coding training for the development team.

The fix a developer can ship today, plus the bigger lesson.

Check your understanding

In the worked report, which section names the **exact payload** ' OR 1=1 -- ?

And which section must **never** show it?

Answer

- **Findings → Evidence** shows the exact payload so it's reproducible.
- The **Executive summary** must never show commands or payloads.
- Same truth, different altitude — pick the right section.

Detail belongs in Findings; plain risk belongs up top.

Assemble + peer review (Day 4)

- Assemble Parts A–C into one **complete sample report** (all five sections + appendix).
- **Peer review (15 min):** swap with a partner; score against the **report rubric**.
- Give one **glow** (strength) and one **grow** (improvement).

Exit ticket: Hand in the rubric you used to score your partner, with your glow + grow.

The report rubric

Criteria	Exemplary (4)	Beginning (1)
Executive summary	Clear, non-technical, accurate risk	Missing
Methodology	Phases described & justified	Missing
Findings	Each has evidence, severity, impact	Missing
Remediation	Specific, actionable	Missing
Communication	Polished, organized, correct terms	Unclear

Day 5

Revise & Submit + Capstone Kickoff

Warm-up

What's **one change** you're making to your report based on yesterday's peer feedback?

Capstone Kickoff

Next week, each **team** will:

1. Run the **full attack lifecycle** on an authorized target:
recon → scan/enumerate → exploit → privilege-escalate → document.
2. Write a **full report** (report rubric).
3. Give a **short presentation** (presentation rubric).

The dress rehearsal you just did is the real thing next week.

Capstone rules

- Targets are **picoCTF** and/or an **approved beginner TryHackMe room** — ONLY.
- **No real, third-party, or out-of-scope systems. Ever.**
- Teams of 2–3; everyone documents.
- You'll get: team assignment, target list, checklist, due dates.

Restate the safety rule: the approved list is the **entire** universe of legal targets.

Why accuracy protects everyone

- **Exaggerate** → the client wastes money on a fake emergency.
- **Hide** → a real hole stays open and customers get hurt.
- Either way you **break the trust** you were paid for.

The client acts on your words. Make them true.

Check your understanding

You found a Critical bug, but it was *embarrassingly* easy — a default password left unchanged.

Do you dress it up or downplay it?

Answer

- **Neither.** Report it accurately as **Critical**, with evidence.
- "Easy" doesn't mean "minor" — a default password is a real door.
- Recommend the fix: change it, enforce a password policy.

Honesty outranks ego. The client's safety comes first.

Team task (Day 5)

- Finish revising and **submit the sample report**.
- With your team, **confirm your target** and write your **capstone scope statement**:

"We are authorized to test only [the approved target our instructor assigned]. No other systems are in scope."

Exit ticket: Submit team name, target, and your one-sentence scope/authorization statement.

Unit recap

- Clients pay for the **report**, not the hack.
- Five sections: **exec summary, methodology, findings, remediation, appendices**.
- Every finding: **evidence + severity + impact**, with **specific remediation**.
- Severity = **likelihood × impact** (no inflation).
- Same truth, **two audiences**, different altitude.
- **Accuracy** is an ethical duty — no exaggerating, no hiding.

Exit Discussion

You found a **Critical** flaw — but it was *embarrassingly* easy (a default password).

You worry the client will think the test was "too simple."

Are you tempted to **dress it up** or **downplay it**? What does honest reporting require, and why does the client's safety outrank your ego?